

Manuel developpeur

DroneRL Environment

Projet INFO3 – Methodes numeriques pour l'Apprentissage Machine

Annee 2025/2026

1 Objectif du manuel

Ce manuel explique comment le projet est organise et comment reutiliser l'environnement avec un futur algorithme d'apprentissage par renforcement.

Le projet ne contient pas encore l'algorithme RL. Il fournit le moteur de simulation, les etats, les actions, les transitions et les modes de test.

2 Organisation du projet

Fichier ou dossier	Role
<code>main.py</code>	menu principal et affichage console
<code>src/config.py</code>	constantes de configuration
<code>src/environment/state.py</code>	classe representant un etat du drone
<code>src/environment/grid.py</code>	grille, voisins, zones et dangers
<code>src/environment/environment.py</code>	moteur principal de l'environnement
<code>src/modes/automatic.py</code>	mode automatique et execution de listes d'actions
<code>src/modes/exhaustive.py</code>	exploration recursive des etats
<code>scenarios/</code>	fichiers CSV de test

3 Configuration

La configuration principale est dans `src/config.py`.

```
MAX_X = 5
MAX_Y = 5
DANGER_SET = {
    (2, 2),
    (3, 3),
    (4, 1),
}

BASE_X = 0
BASE_Y = 0

BATTERY_MAX = 100
BATTERY_INITIAL = 100

MOVE_COST = 1
AVOID_COST = 2
CHARGE_STEP = 5

ZONE_COUNT = 4
```

Pour changer la taille de la grille, la base, les dangers ou les couts de batterie, il faut modifier ce fichier.

4 Etat du drone

L'etat du drone est defini dans `src/environment/state.py`.

Il contient :

- `pos_x, pos_y` : position du drone ;
- `base_x, base_y` : position de la base ;
- `battery` : batterie restante ;
- `obstacle_distance` : 0 si danger immediat, 2 sinon ;
- `docked` : vrai si le drone est arrime ;
- `airborne` : vrai si le drone est en vol ;
- `crashed` : vrai si le drone est crashe ;
- `visited_zones` : ensemble des zones visitees.

La methode `copy()` permet de copier un etat sans modifier l'etat original.

5 Moteur de l'environnement

La classe principale est `Environment`, dans `src/environment/environment.py`.
Les methodes principales sont :

Methode	Role
<code>reset()</code>	remet l'environnement a l'etat initial
<code>get_state()</code>	retourne une copie de l'etat courant
<code>available_actions(state)</code>	donne les actions executables
<code>next_states(action, state)</code>	donne tous les successeurs possibles
<code>apply_action(action, selector)</code>	applique un successeur choisi
<code>check_invariant(state)</code>	verifie la coherence d'un etat
<code>check_crash(state)</code>	declenche le crash si danger immediat

6 Actions disponibles

Les actions implementees sont :

- `takeoff` : decollage ;
- `move_step` : deplacement cardinal non deterministe ;
- `avoid_maneuver` : eviter les cases dangereuses si possible ;
- `return_home` : retour progressif vers la base ;
- `dock_cmd` : arrimage a la base ;
- `undock_cmd` : desarrimage ;
- `charge_step` : recharge de la batterie ;
- `emergency_stop` : arret d'urgence et crash.

Des alias courts existent aussi dans `Environment.ACTION_ALIASES`, par exemple `m` pour `move_step`.

7 Preconditions et transitions

Chaque action verifie ses preconditions avant de produire un nouvel etat.

Exemples :

- `move_step` demande que le drone soit en vol, non arrime, non crashe et avec une batterie positive ;
- `dock_cmd` demande que le drone soit en vol et a la position de la base ;
- `charge_step` demande que le drone soit arrime ;
- `avoid_maneuver` demande une batterie strictement superieure a 1.

Si une transition arrive sur une case de `DANGER_SET`, alors `obstacle_distance` devient 0 et `check_crash()` declenche `emergency_stop()`.

8 Non-determinisme

Le non-determinisme est gere par la methode `next_states()`.

Exemple :

```
from src.environment.environment import Environment

env = Environment()
env.apply_action("takeoff")

successors = env.next_states("move_step")
print(len(successors))
```

Pour `move_step`, plusieurs successeurs peuvent exister, car le drone peut aller vers plusieurs voisins valides.

Dans les modes interactif et automatique, un successeur est choisi aléatoirement. Dans le mode exhaustif, tous les successeurs sont explorés.

9 Utiliser l'environnement avec un futur agent RL

Un agent RL peut utiliser l'environnement sans passer par l'interface console.

Exemple simple :

```
from src.environment.environment import Environment

env = Environment()
state = env.get_state()

while not state.crashed:
    actions = env.available_actions(state)
    action = agent.choose_action(state, actions)
    successors = env.next_states(action, state)

    if not successors:
        break

# Exemple simple : choix du premier successeur
state = successors[0]
```

Dans un vrai algorithme RL, il faudra ajouter une fonction de recompense. Cette partie n'est pas implementee ici, car le sujet demande d'abord l'environnement.

10 Mode automatique pour les tests

Le mode automatique est implemente dans `src/modes/automatic.py`.

Il peut etre utilise avec une liste Python :

```
from src.environment.environment import Environment
from src.modes.automatic import AutomaticMode

env = Environment()
mode = AutomaticMode(env)

mode.run_automatic_mode([
    "takeoff",
    "move_step",
    "return_home",
    "dock_cmd"
])
```

Il peut aussi etre utilise avec un fichier CSV grace a la methode `run(csv_file)`.

11 Mode exhaustif

Le mode exhaustif est implemente dans `src/modes/exhaustive.py`.

Il suit l'algorithme recursive :

```
Explore(s, S, maxDepth, maxBreadth)
```

La methode principale est :

```
from src.environment.environment import Environment
from src.modes.exhaustive import ExhaustiveMode

env = Environment()
mode = ExhaustiveMode(env)
```

```
stats = mode.run(max_depth=3, max_breadth=-1)
print(stats)
```

La valeur -1 signifie que la limite est desactivee.

12 Invariants

Les invariants sont verifiees dans `check_invariant()`.

Les principaux invariants sont :

- la position du drone reste dans la grille ;
- la batterie reste entre 0 et 100 ;
- si `crashed = True`, alors `airborne = False` ;
- si `docked = True`, alors le drone est a la base ;
- si `docked = True`, alors `airborne = False`.

13 Compiler et verifier

Pour verifier que les fichiers Python se compilent :

```
python3 -m py_compile main.py src/config.py src/environment/environment.py src/
environment/grid.py src/environment/state.py src/modes/automatic.py src/modes/
exhaustive.py
```

Pour lancer le programme :

```
python3 main.py
```

14 Sources

Ce manuel est base sur :

- l'enonce du projet fourni dans `projet.pdf` ;
- le complement de l'enonce sur l'exploration recursive ;
- le code source du projet : `main.py`, `src/`, `scenarios/`.